Duyệt Đồ Thị Bằng DFS, BFS và Kiểm tra Đồ thị Liên Thông

Lê Văn Hoàng An

Ngày 23 tháng 3 năm 2026

Mục lục

BFS

1. Giới thiệu
2. Minh họa
3. Lý thuyết
4. Cài đặt
5. Ví dụ
6. Animation
7. Ứng dụng

DFS

1. Giới thiệu
2. Minh họa
3. Lý thuyết
4. Cài đặt
5. Ví dụ
6. Animation
7. Ứng dụng

Đồ thị liên thông

1. Khái niệm
2. Kiểm tra liên thông
3. Liên hệ BFS
4. Đếm thành phần
5. Minh họa
6. Cài đặt

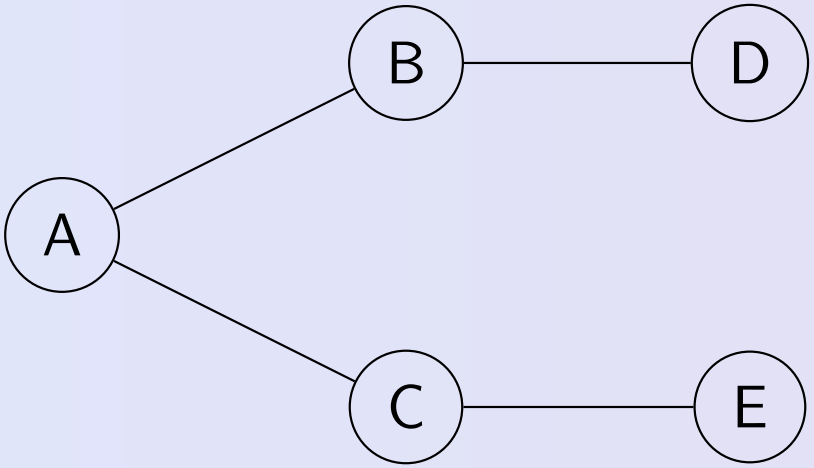
Đồ thị là gì?

- Đồ thị gồm tập đỉnh (Vertex) và cạnh (Edge)
- Biểu diễn mối quan hệ giữa các đối tượng
- Có thể là đồ thị vô hướng hoặc có hướng
- Ứng dụng: mạng xã hội, bản đồ, AI, hệ thống gợi ý

Ví dụ thực tế:

- Facebook: người dùng = đỉnh, kết bạn = cạnh
- Google Maps: địa điểm = đỉnh, đường đi = cạnh

Minh họa đồ thị và BFS



- ▶Bắt đầu từ A
- 📚BFS duyệt theo từng lớp
- ➔A → B, C
- ➔Tiếp → D, E

Ý tưởng + Giả mã BFS

- Mục tiêu: duyệt theo từng lớp (level)
- Cấu trúc: Queue (FIFO)
- Mỗi đỉnh thăm đúng 1 lần

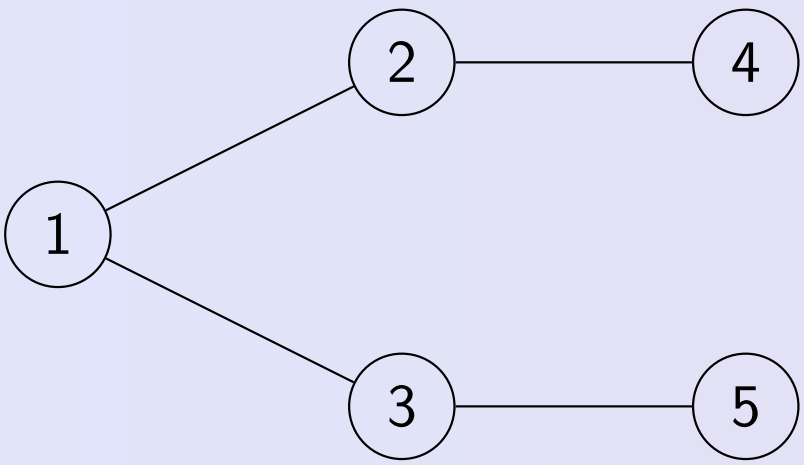
Quy trình thực hiện:

1. Khởi tạo:
   - Đánh dấu tất cả là chưa thăm
   - Đưa đỉnh start vào queue
2. Lặp:
   - Lấy đỉnh đầu u
   - Xử lý u
   - Duyệt các đỉnh kề v
   - Nếu chưa thăm $\rightarrow$ thêm vào queue

Code BFS (C++)

```
#include <bits/stdc++.h>
using namespace std;

void BFS(int start, vector<vector<int>>& adj) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        // Xu ly dinh u
        cout << u << "␣";

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
/*
- Thoi gian: O(V + E)
- Duyet theo tung lop
- Dung trong tim duong ngan nhat
*/
```

Ví dụ minh họa BFS

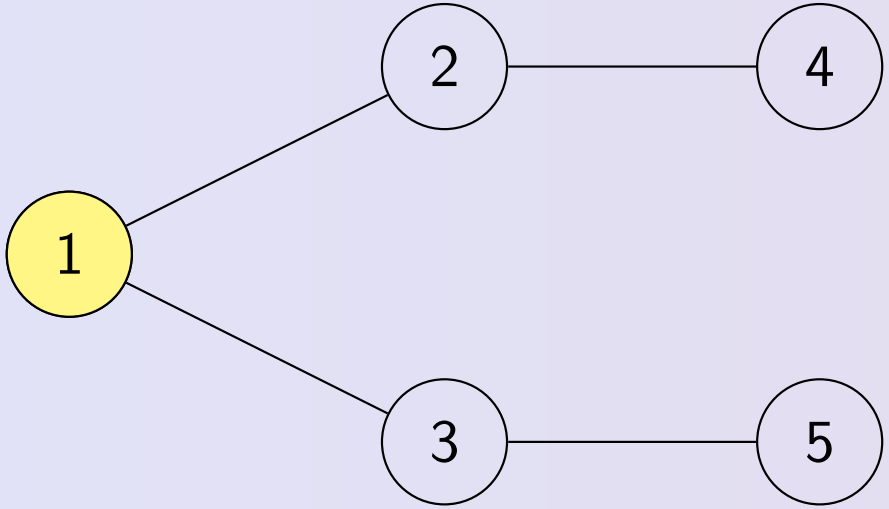


- Thứ tự:

$$1 \to 2 \to 3 \to 4 \to 5$$

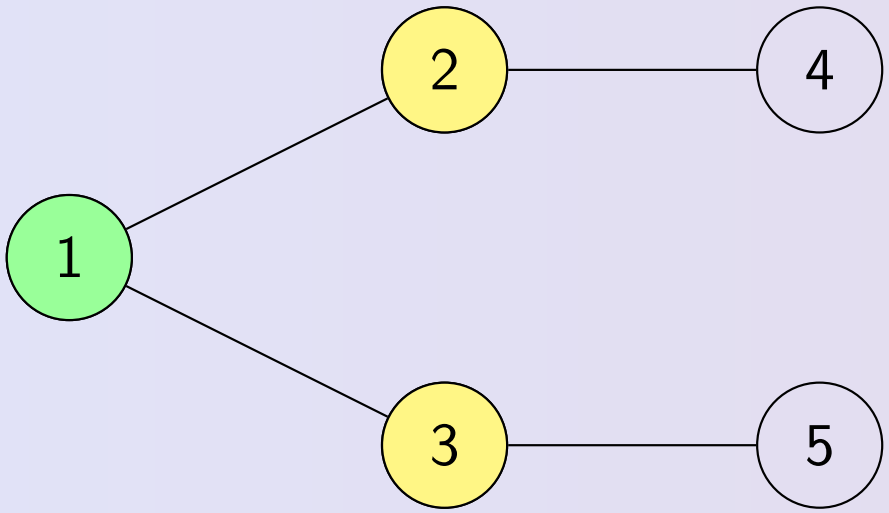
- Lớp:
  - L0: 1
  - L1: 2, 3
  - L2: 4, 5

Minh họa BFS từng bước



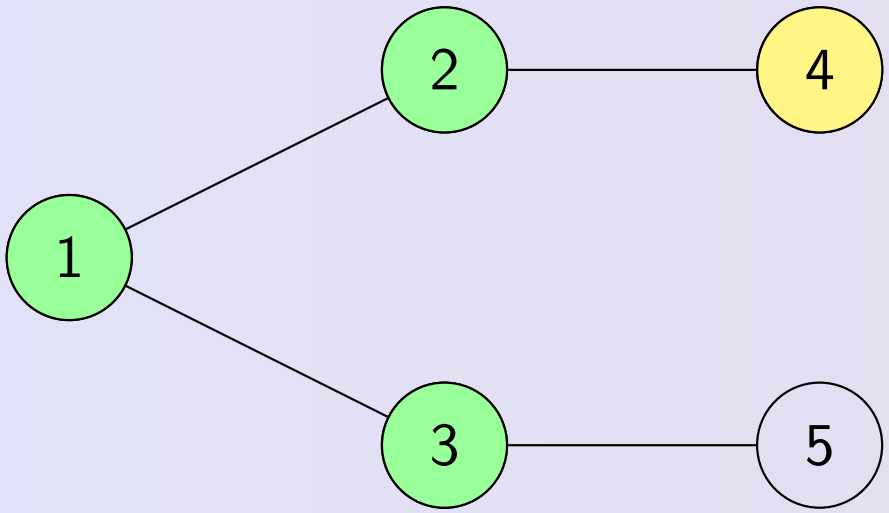
Bắt đầu từ đỉnh 1

Minh họa BFS từng bước



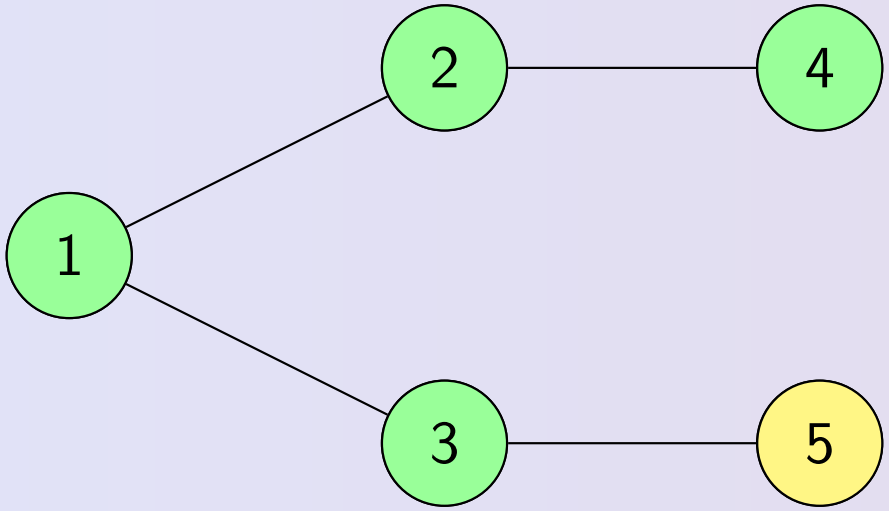
Thêm 2, 3 vào queue

Minh họa BFS từng bước



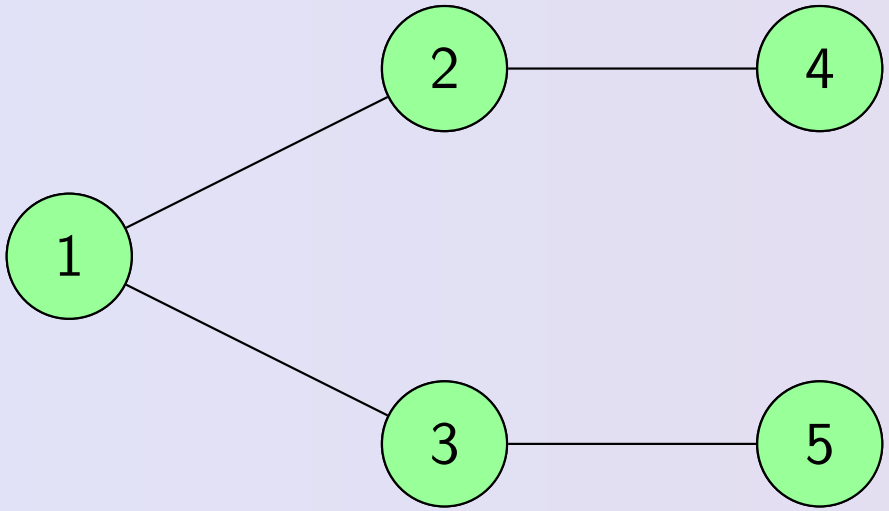
Xử lý 2 → thêm 4

Minh họa BFS từng bước



Xử lý 3 $\rightarrow$ thêm 5

Minh họa BFS từng bước



Hoàn tất BFS

Ứng dụng của BFS

- Tìm đường đi ngắn nhất (không trọng số)
- Kiểm tra liên thông
- AI, game, mê cung
- Mạng xã hội (gợi ý bạn bè)

Kết luận

- BFS là thuật toán nền tảng
- Độ phức tạp tối ưu $O(V + E)$
- Rất quan trọng trong thực tế

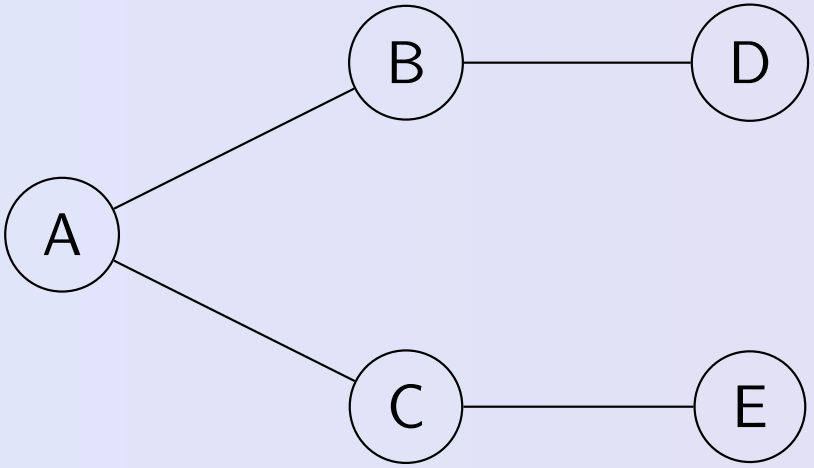
DFS là gì?

- 🔍DFS = Depth-First Search (Duyệt theo chiều sâu)
- ↓Đi sâu nhất có thể trước khi quay lui
- ↺Backtracking khi không còn đường đi
- ⚙Sử dụng đệ quy hoặc stack

Ý nghĩa:

- Giống khám phá mê cung
- Đi một hướng đến cùng rồi quay lại

Minh họa DFS



- ▶Bắt đầu từ A
- →Đi sâu: A → B → D
- ↺Quay lui → A → C → E

Ý tưởng + Giả mã DFS

- Mục tiêu: duyệt sâu theo nhánh
- Cấu trúc: stack / đệ quy
- Mỗi đỉnh chỉ thăm 1 lần

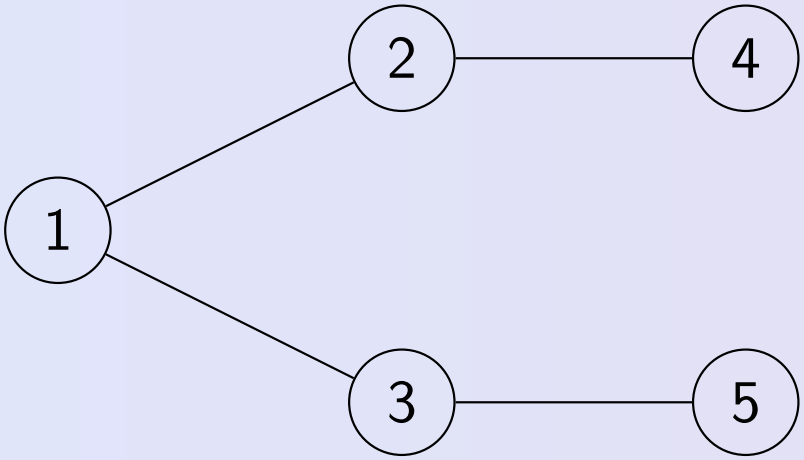
Quy trình:

1. Đánh dấu đỉnh hiện tại
2. Xử lý đỉnh
3. Duyệt các đỉnh kề chưa thăm
4. Gọi DFS đệ quy

Code DFS (C++)

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<bool> visited;

void dfs(int u) {
    visited[u] = true;

    // Xu ly dinh u
    cout << u << "␣";

    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}

/*
Phan tich:
- Thoi gian: O(V + E)
- Moi dinh duyet 1 lan
- Su dung de quy don gian
*/
```

Ví dụ minh họa DFS

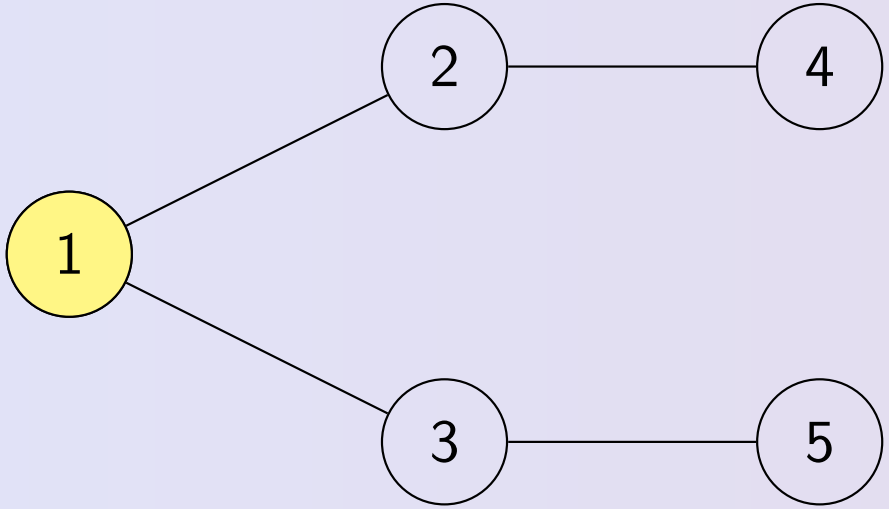


- Thứ tự:

$$1 \to 2 \to 4 \to 3 \to 5$$

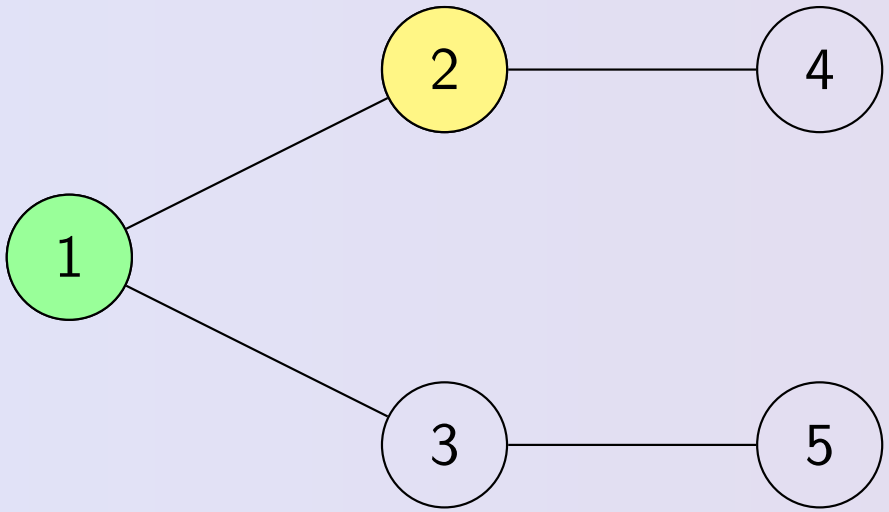
- Có backtracking
- Không theo lớp như BFS

Minh họa DFS từng bước



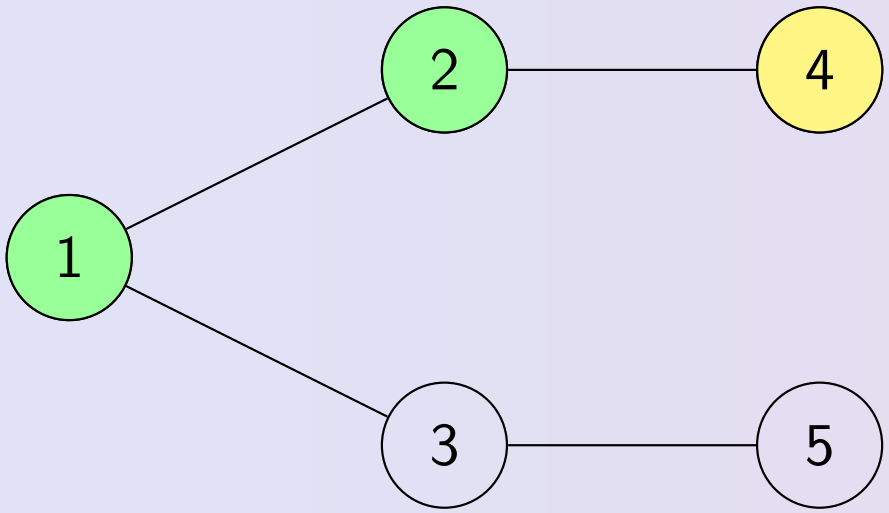
Bắt đầu từ 1

Minh họa DFS từng bước



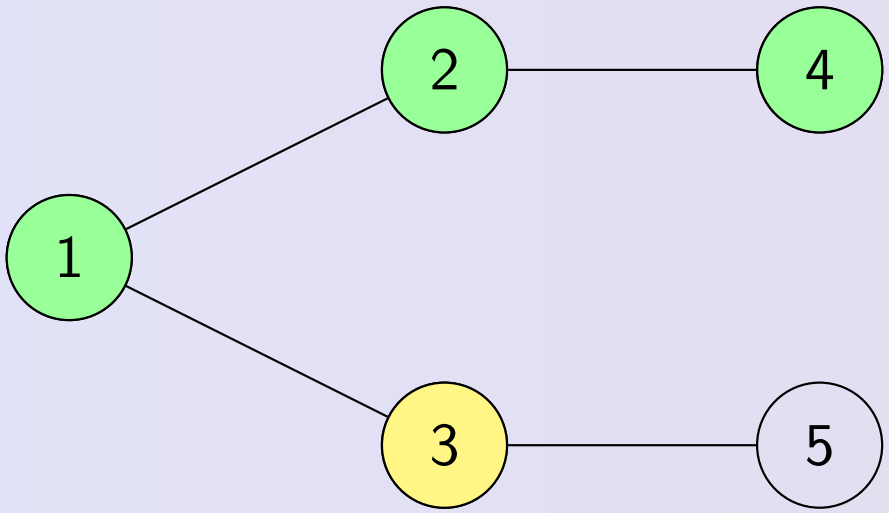
Đi sâu đến 2

Minh họa DFS từng bước



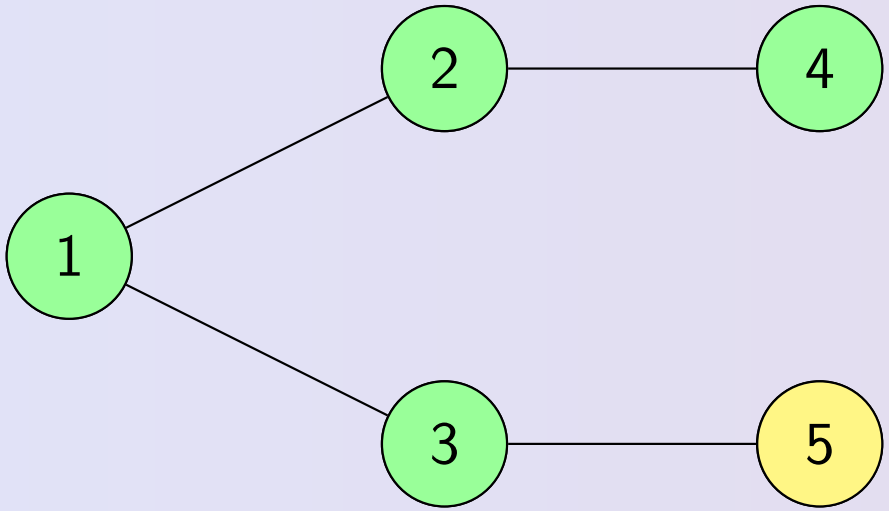
Tiếp tục đến 4

Minh họa DFS từng bước



Quay lui $\rightarrow$ sang 3

Minh họa DFS từng bước



Hoàn tất DFS

Ứng dụng của DFS

- Tìm thành phần liên thông
- Kiểm tra chu trình
- Backtracking, AI, game

Kết luận

- DFS là thuật toán nền tảng
- Độ phức tạp tối ưu $O(V + E)$
- Quan trọng trong nhiều bài toán nâng cao

Khái niệm và kiểm tra liên thông

Khái niệm đồ thị liên thông

- Đồ thị liên thông: tồn tại đường đi giữa mọi cặp đỉnh
- Nếu không → chia thành nhiều thành phần
- Mỗi thành phần là một nhóm liên thông riêng biệt

Kiểm tra liên thông

- Chọn một đỉnh bất kỳ
- Duyệt bằng BFS hoặc DFS
- Thăm hết → liên thông
- Còn đỉnh chưa thăm → không liên thông

Liên hệ BFS và đếm thành phần

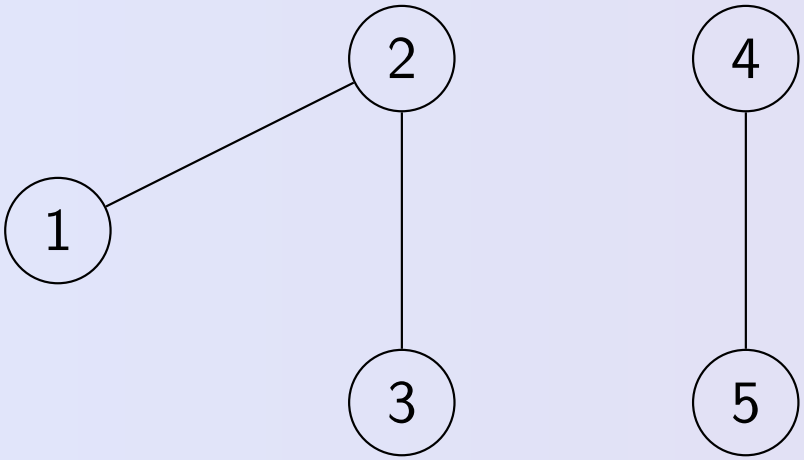
Liên hệ BFS

- BFS duyệt theo từng lớp (level)
- Phù hợp tìm đường đi ngắn nhất
- Độ phức tạp: $O(V + E)$

Đếm thành phần liên thông

- Duyệt toàn bộ các đỉnh
- Mỗi lần DFS/BFS $\rightarrow$ 1 thành phần
- Kết quả = số thành phần

Ví dụ đồ thị không liên thông



- Thành phần 1: 1, 2, 3
- Thành phần 2: 4, 5
- ✗Không có cạnh nối
- ⚠→ Không liên thông

Code DFS kiểm tra liên thông

```
vector<vector<int>> adj;
vector<bool> da_tham;

void dfs_lt(int u) {
    da_tham[u] = true;
    for (int v : adj[u]) {
        if (!da_tham[v]) dfs_lt(v);
    }
}

bool kt_lt(int n) {
    da_tham.assign(n, false);
    dfs_lt(0);

    for (int i = 0; i < n; i++) {
        if (!da_tham[i]) return false;
    }
    return true;
}
```

Đếm thành phần liên thông

```
int dem_tplt(int n) {
    da_tham.assign(n, false);
    int dem = 0;

    for (int i = 0; i < n; i++) {
        if (!da_tham[i]) {
            dfs_lt(i);
            dem++;
        }
    }
    return dem;
}
```

Kết luận

- BFS: duyệt theo lớp tìm đường đi ngắn nhất
- DFS: kiểm tra liên thông
- Ứng dụng rộng trong thực tế

Cảm ơn!